

TRACE

Leak-Proof, Traceable Exam Papers

*Make an exam paper impossible to open before the exam —
and traceable to its exact source if it ever leaks.*

India's FAR AWAY 2026 • Theme: Secure, Leak-Proof Examinations

Python • FastAPI • SQLite • React • Vite • Tailwind
AES-256-GCM • Shamir 3-of-5 (from scratch) • DCT watermarking • SHA-256 hash chain

Contents

1. Executive Summary — what Trace is, in one minute
2. The Problem It Solves
3. The Three Guarantees
4. How It Works, End to End (the demo story)
5. System Architecture
6. Technology Stack
7. Guarantee 1 — Multi-Custodian, Time-Locked Decryption
8. Guarantee 2 — Invisible Per-Candidate Watermarking
9. Guarantee 3 — Tamper-Evident Audit Log
10. Defense in Depth — Dynamic Assembly & Leak-Match Detector
11. The Backend — Modules & Data Model
12. The API — Every Endpoint
13. The Frontend — Four Role Dashboards
14. Security Model & Trust Boundary
15. Testing & Verification
16. How to Run It Yourself
17. Build Status (Milestones)
18. Project Layout
19. Glossary

1. Executive Summary

Exam paper leaks are a recurring, high-impact failure of trust. A single insider with early access can compromise an entire exam for millions of candidates. Trace is a working system that removes that possibility and, if a leak somehow happens through a screen photo or printout, traces the leaked copy back to the exact candidate it was issued to.

Trace rests on three real security guarantees — implemented in code and proven by tests, not slideware:

- **Sealed until the moment:** the paper cannot be opened by any single person, and cannot be opened before the exam start time;
- **Traceable forever:** every printed/displayed copy carries an invisible fingerprint identifying its candidate;
- **Tamper-evident:** every access is recorded in a log that cannot be silently altered.

The system is a full-stack application: a Python/FastAPI backend implementing the cryptography from first principles, and a React dashboard that lets the four real-world roles — Exam Controller, Custodian, Candidate, and Investigator — each play their part in a live, end-to-end demonstration.

2. The Problem It Solves

High-stakes exams (entrance tests, recruitment, board exams) depend on the paper staying secret until the exam begins. In practice two things go wrong:

- **The insider leak** — Somewhere between the printer and the exam hall, one trusted person opens the paper early and sells or shares it.
- **The accountability gap** — Even with secrecy, once a copy is out there is no way to know whose copy it was, so nobody is ever held accountable.

Trace attacks both: it makes early opening cryptographically impossible (no single insider holds the key, and a server clock blocks decryption until release time), and it makes every copy individually identifiable so a leak names its own source.

3. The Three Guarantees

Guarantee	What it means	How it is achieved
1. Just-in-time, multi-custodian decryption	No single insider — and no pair — can open the paper, and nobody can open it before exam start.	AES-256-GCM encryption; the key is split with Shamir's Secret Sharing (3-of-5) across independent custodians, plus a server-enforced release-time gate.
2. Per-candidate invisible watermarking	Any leaked image traces back to its exact source candidate.	An invisible DCT-domain fingerprint encoding center ID +

		candidate ID is embedded in each rendered copy; it survives JPEG recompression.
3. Tamper-evident audit log	Altering any past event is detectable.	An append-only SHA-256 hash-chained log of every access event; changing any entry breaks the chain.

4. How It Works, End to End

The whole system can be experienced as one continuous story, with each role acting in turn:

Act 1 — Seal (the Exam Controller / Admin)

The controller uploads a paper and seals an exam. Behind the scenes the backend generates a fresh 256-bit key, encrypts the paper with AES-256-GCM, then splits the key into 5 custodian shares using Shamir's Secret Sharing — any 3 of which can rebuild it. The plaintext key is never stored. A release time is set (e.g. "+30 seconds" for the demo).

Act 2 — Authorize (the five Custodians)

Each custodian sees their own key share — masked until the release window opens. When the window opens, each submits their share. The dashboard shows a live quorum: "2 of 3 required shares." Crucially, even when all three valid shares are present, the server refuses to decrypt until the clock passes release time.

Act 3 — Unlock (the time gate releases)

Once both conditions hold — enough shares AND release time reached — the server reconstructs the key by Lagrange interpolation, decrypts the paper (the GCM tag proves the reconstruction was correct), and re-wraps the key under a server secret so candidates can be served without the raw key ever leaving the server. The vault opens, live.

Act 4 — Read (the Candidate)

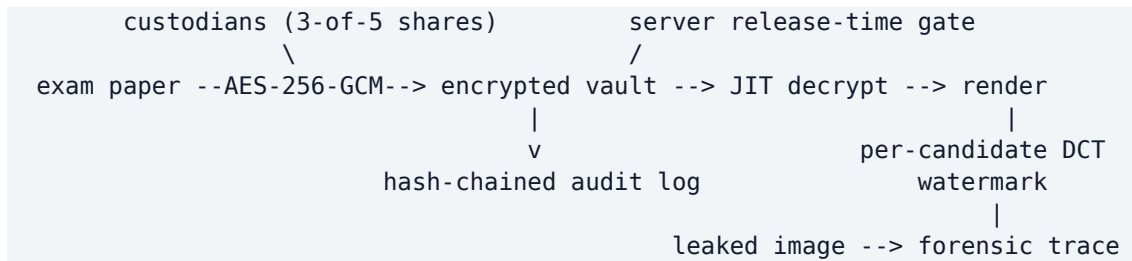
Before release the candidate sees only a countdown. After unlock they receive their own copy of the paper — rendered to an image and stamped with an invisible watermark unique to them. To the eye it is identical to everyone else's (PSNR ~36 dB); under the hood it carries their fingerprint.

Act 5 — Trace & Verify (the Investigator)

If a copy leaks, the investigator uploads the leaked image — even a re-compressed JPEG screen photo. Trace extracts the fingerprint and matches it against the issue registry, naming the exact candidate with a confidence score. Finally, the investigator clicks "Verify Integrity" to recompute the entire SHA-256 audit chain and confirm no record was tampered with.

5. System Architecture

The data flow, from sealed paper to forensic trace:



The backend is the single trusted enforcement point: it holds custodian shares in escrow, enforces the time gate, performs reconstruction and decryption, embeds watermarks, and maintains the audit chain. The frontend is a thin client — every animation it shows is driven by a real backend event, never mock data.

6. Technology Stack

Layer	Technology	Role
Backend framework	Python 3.10+ · FastAPI	REST API, request validation, OpenAPI docs
Database / ORM	SQLite · SQLAlchemy 2.0	Exams, users, shares, watermark registry, audit events
Cryptography	pycryptodome (AES-GCM) + from-scratch GF(256) Shamir	Encryption and secret sharing
Auth	PyJWT (HS256) · OAuth2 password flow	4 roles: admin / custodian / candidate / investigator
Watermarking	OpenCV (DCT) · NumPy · Pillow	Invisible fingerprint embed / extract / trace
Frontend	React 18 · Vite 5 · Tailwind 3	Four role dashboards, dark mission-control theme
Frontend data/UX	TanStack React Query 5 · Framer Motion 11	Live polling of real endpoints; animations
Testing	pytest	~816 lines across 8 test files, full crypto + API coverage

7. Guarantee 1 — Multi-Custodian, Time-Locked Decryption

AES-256-GCM encryption

- **Key:** 256-bit (32-byte) key, generated fresh per exam with `os.urandom`.
- **Nonce:** 96-bit (12-byte) nonce — the GCM-recommended size.
- **Tag:** 128-bit (16-byte) authentication tag — detects any tampering with the ciphertext.
- **AAD:** Associated data binds the ciphertext to its context: `"center={id};exam={name}"`.

Stored layout: nonce (12) || tag (16) || ciphertext. The plaintext key is never written to disk.

Shamir's Secret Sharing over GF(256) — built from scratch

Rather than calling a black-box library, the secret sharing is implemented from first principles over the finite field GF(2⁸), the same field AES itself uses:

- **Field:** The AES irreducible polynomial 0x11B ($x^8 + x^4 + x^3 + x + 1$) with generator 0x03.
- **Arithmetic:** Addition is XOR; multiplication, division and inversion are O(1) via precomputed 512-entry exp and 256-entry log tables.
- **Split:** The 32-byte key is split byte-by-byte. Each byte becomes the constant term of a random degree $k-1$ polynomial; shares are evaluations $f(x)$ at $x = 1..n$.
- **Reconstruct:** Any k shares rebuild each secret byte via Lagrange interpolation at $x = 0$. In GF(256) negation is identity, so subtraction is also XOR.
- **Security:** k shares always reconstruct; $k-1$ shares leave the secret uniformly random — information-theoretically secure, not just computationally hard.

The demo default is 3-of-5: no single custodian and no pair can rebuild the key; it takes three distinct custodians authorizing together.

The server-enforced time gate

Reconstruction is gated by two independent guards. Even when all three valid shares are submitted, the server refuses to decrypt until the wall-clock passes `release_time`. This guard is unconditional — there is no override — and every blocked attempt is logged as an UNLOCK_DENIED audit event with the seconds remaining.

8. Guarantee 2 — Invisible Per-Candidate Watermarking

Each candidate's paper is rendered to an image and stamped with an invisible fingerprint in the frequency domain, using a Koch-Zhao differential DCT scheme.

What is embedded

- A 16-bit MAGIC marker (0xACE5) used to detect whether a watermark is present at all.
- A 64-bit fingerprint = BLAKE2b-64 of (center ID, exam:candidate code) — unique per candidate per exam. Total payload: 80 bits.

How it is embedded

- The image luminance is split into 8x8 DCT blocks (the same block size JPEG uses).
- Two mid-band coefficients at positions (2,3) and (3,2) are nudged so their relative order encodes one payload bit, with an enforced gap of 26 between them.
- Payload bits are repeated across hundreds of blocks, then an inverse DCT rebuilds the image.

The result is imperceptible to the eye — PSNR around 36 dB — yet robust.

Why it survives a leak

A real leak is rarely the original file — it is a screen photo or a re-saved JPEG. The watermark survives because it encodes the relative order of mid-band coefficients (which JPEG quantization preserves far better than absolute values) and because each bit is spread across many blocks, so a few destroyed blocks cannot flip the aggregate decision. Extraction uses a soft decision: it sums the coefficient differences across all blocks and reads the sign.

Tracing a leak

The investigator's upload is run through extraction, and the recovered 64-bit fingerprint is matched against the registry of issued watermarks by Hamming distance (tolerating up to 2 flipped magic bits). The output reports whether a watermark was present, the matched candidate, the bit distance, and a confidence score = $1 - (\text{bit_distance} / 64)$.

9. Guarantee 3 — Tamper-Evident Audit Log

Every meaningful action is recorded as an event in an append-only log, and each event is chained to the one before it by a SHA-256 hash — the same idea that makes a blockchain tamper-evident.

Each event's hash is computed over its own fields plus the previous event's hash:

```
hash(i) = SHA256( id | timestamp | actor | action | target | details |  
prev_hash )
```

- The chain starts from a fixed genesis hash (64 zeros).
- Recorded actions include: LOGIN, EXAM_SEALED, SHARE_SUBMITTED, UNLOCK_DENIED, PAPER_UNLOCKED, PAPER_ACCESSED, WATERMARK_ISSUED, LEAK_TRACED, AUDIT_VERIFIED, QUESTION_ADDED, PAPER_ASSEMBLED, and LEAK_MATCHED.
- Verification recomputes every hash and checks every prev_hash link. If anyone edits, inserts, or deletes a row, the recomputed hash stops matching and the break is reported with the exact event id.
- A threading lock serialises appends so concurrent writes cannot corrupt the tail (a real concurrency race that was caught and fixed during development).

10. Defense in Depth — Dynamic Assembly & Leak-Match Detector

Beyond the three core guarantees, Trace adds two further layers that directly shrink the chance and the blast radius of a leak — and trace one even when no image exists.

10.1 Dynamic per-candidate paper assembly

A static exam is one secret blob: leak it once and the whole exam is compromised, for everyone. Dynamic assembly removes that single point of failure. Instead of one paper, the system keeps a large bank of individually-encrypted questions; at release time a blueprint (sections + counts)

and a per-candidate seed select a unique subset and assemble that candidate's paper on demand.

- **Encrypted bank** — Each question body is AES-256-GCM ciphertext at rest, under a server question-bank key that is domain-separated from the vault key — a database dump cannot read the bank.
- **Unpredictable** — The selection seed is HMAC(exam_key, "exam:candidate:section"). Because the exam key only exists after the Shamir quorum AND the time gate open, no setter, admin, or database reader can predict which questions reach which candidate before release.
- **Low blast radius** — Every candidate's paper is content-distinct, so a leaked copy exposes only one variant — and, combined with the DCT watermark, still names its owner.
- **Immutable once sealed** — At seal time the per-section pools are frozen inside the encrypted manifest, so retiring or editing a bank question later cannot change (or break) an already-sealed exam's papers.
- **Never stored** — Only the selection (a list of question ids) is recorded per candidate — the assembled paper is never persisted in plaintext; it exists only in memory while one request is served.

10.2 Leak-match detector (text-only leaks)

Real leaks are often just text — a question pasted into a chat group, with no image to trace. The leak-match detector handles exactly this case. An investigator pastes the suspected text and Trace:

- **Identifies what leaked** — matches it against the encrypted bank by token-containment on each question's prompt (option labels are excluded so short prompts are not over-matched), flagging every question the text contains;
- **Narrows who leaked it** — intersects the per-candidate selection records: a candidate whose paper contained every matched question is a prime suspect; otherwise the highest-overlap candidate is the strongest lead. The issued-watermark fingerprint is attached for an image follow-up.

This is where dynamic assembly pays off forensically: because every candidate received a different combination, a leak containing several questions intersects to a very small suspect set — often a single candidate.

11. The Backend — Modules & Data Model

Module map

Module	Responsibility
crypto/	gf256, shamir, aes_gcm — the from-scratch cryptographic core
security/	passwords (bcrypt), tokens (JWT), deps (role

	guards), keywrap (server key wrapping)
audit/	chain.py — SHA-256 hash-chained event log + verification
watermark/	core (DCT embed/extract), render (text -> image), trace (registry match)
services/	exams.py (seal/unlock/serve), assembly.py (dynamic question bank), leakmatch.py (text leak detector), watermarking.py
api/	app.py (FastAPI factory) + routers/ (auth, exams, questions, investigator, audit)
models.py / schemas.py / db.py	SQLAlchemy tables, Pydantic request/response shapes, DB session
bootstrap.py / config.py	demo seeding + idempotent migration; env-driven settings

Data model (tables)

Table	Key fields	Purpose
users	username, role, password_hash, candidate_code, center_id	Accounts for the four roles
exams	name, release_time, threshold_k, nonce, tag, ciphertext, aad, status, released_key_wrapped, assembly_mode, blueprint	A sealed paper/manifest and its unlock state
custodian_shares	exam_id, custodian_id, x, y	Each custodian's escrowed Shamir share
share_submissions	exam_id, custodian_id, submitted_at	Authorizations during the unlock ceremony
questions	subject, section, topic, difficulty, nonce, tag, ciphertext, contributor	The encrypted question bank (bodies are ciphertext)
candidate_papers	exam_id, username, selected_question_ids, assembled_at	Per-candidate selection (ids only, never plaintext)
issued_watermarks	exam_id, username, fingerprint, center_id, candidate_code	Registry for tracing leaked images
audit_events	timestamp, actor, action, target, details, prev_hash, hash	The tamper-evident hash chain

12. The API — Every Endpoint

Method	Path	Role	Purpose
POST	/auth/login	any	OAuth2 password login -> JWT
GET	/auth/me	any	Current user profile
POST	/exams	admin	Seal a paper/manifest (encrypt + Shamir-split the key)
GET	/exams	any	List all exams

GET	/exams/{id}	any	Inspect one exam's metadata
GET	/exams/{id}/unlock/status	any	Live unlock progress (drives the vault UI)
GET	/exams/{id}/blueprint	any	Dynamic exam structure (counts, not content)
GET	/exams/{id}/my-share	custodian	Fetch own share (masked until window opens)
POST	/exams/{id}/shares/submit	custodian	Authorize release of one share
GET	/exams/{id}/paper	candidate/admin/investigator	Read released paper (assembled per-candidate if dynamic)
GET	/exams/{id}/paper/image	candidate/admin	Per-candidate watermarked PNG
POST	/questions	admin	Add one AES-GCM-encrypted question to the bank
GET	/questions	admin	List bank metadata only (never bodies)
POST	/investigator/trace	investigator/admin	Trace a leaked image to a candidate
POST	/investigator/match	investigator/admin	Match leaked text to bank + narrow the source
GET	/audit	investigator/admin	The append-only event log
GET	/audit/verify	investigator/admin	Recompute the hash chain -> intact?

Authentication is a JWT bearer token (HS256) issued at login, carrying the username and role, with a configurable lifetime (default 12 hours). Every protected route checks the role; missing/invalid tokens get 401, insufficient role gets 403.

13. The Frontend — Four Role Dashboards

A single dark "security operations center" web app hosts all four roles. A floating role-switcher (bottom center) jumps between them instantly — each identity is a real JWT session pre-authenticated against the backend, so there is no mock data anywhere.

Admin (Exam Controller)

Seal an exam (name, subject, center, threshold, release window), browse sealed exams, and watch a live operations panel: the vault animation, the share quorum, the encryption badge, and the time-gate countdown — all polled from the real backend every 1.5 seconds. A Static/Dynamic toggle switches the seal form to a blueprint editor for dynamic exams, and the operations panel shows a live "dynamic assembly" strip (bank pool + per-candidate count).

Custodian

See your own key share (rendered as a masked pixel grid until the window opens, then revealed as hex), submit it with one action, and watch the shared quorum fill toward the 3-of-5 threshold.

Candidate

Before release: a countdown or an "awaiting shares" progress bar. After unlock: your own invisibly-watermarked paper image, with your fingerprint shown and a download button.

Investigator

Three tools in one console: a Leak-Match Detector (paste suspected leaked text to see the matched bank questions with containment scores and a ranked suspect list); a Forensic Trace (drag-drop a leaked image to trace it to a candidate, with confidence bar and bit distance); and the Audit Ledger — click "Verify Integrity" to recompute the SHA-256 chain and confirm it is intact, with any broken events highlighted in red.

Design system

A small set of reusable primitives (StatusPill, MonoReadout, Card, Metric, CountdownTimer, ShareSlots, VaultState, LogView) give the app a consistent mission-control look: cyan for secure, amber for pending/time-locked, red for denied/leak, green for verified, with glass morphism surfaces and JetBrains Mono / Space Grotesk typography.

14. Security Model & Trust Boundary

Trace defends against the two threats that cause real exam leaks:

- **A single malicious insider** — No one custodian (or pair) can open a paper. The key only exists after k distinct custodians authorize AND the server reconstructs it via Shamir. The key is never stored whole.
- **Early opening** — The release-time gate is enforced server-side and is unconditional: even with every valid share present, decryption is refused until `release_time`.

In this single-node demo the server is the trusted enforcement point and holds custodian shares in escrow; the released key is wrapped under a server secret (derived from the app secret, not stored raw) so a raw database dump cannot decrypt a sealed paper. Hardening against a fully compromised server — HSM-held or client-custodied shares — is called out honestly as future work rather than faked.

Threat Model: Direct Database Access

A natural question is: what happens if an attacker gains direct read access to the database and tries to "open the vault" there? The answer separates the encrypted vault from the escrowed shares.

- **The vault stays sealed** — Reading the exams table yields only the AES-256-GCM encrypted paper (nonce, tag, and ciphertext). The plaintext paper and the AES key are never stored in the vault.
- **No usable key in either state** — Before release, no exam decryption key exists in the database at all. After a legitimate unlock, the released key is stored only in wrapped form, encrypted under a server-held secret derived from TRACE_SECRET_KEY that is maintained outside the database (an environment variable). Consequently, a database dump alone is insufficient to recover exam plaintext in either state.
- **Tampering is detected** — AES-GCM authentication ensures that any modification of stored ciphertext is detected during decryption, so a forged or altered vault is rejected rather than silently served.
- **The remaining soft spot** — The primary remaining risk lies in the escrowed Shamir shares stored in the custodian_shares table: an attacker with unrestricted access to the entire database could recover enough shares to reconstruct the key, bypassing the application-layer quorum and release-time controls. Mitigating this requires moving shares into external custodial systems, HSMs, or trusted execution environments rather than relying on application-level controls alone.

In summary, direct access to the encrypted vault does not reveal exam content — this is a cryptographic guarantee, not an application-level one. The remaining dependency is on trusted server-side share storage, which the production hardening above removes: under external, HSM-, or enclave-held shares, a database compromise alone would not be enough to recover an exam paper.

15. Testing & Verification

Correctness is proven by an automated test suite — 81 tests passing — covering the parts that matter most:

Area	What is proven
GF(256) field	Field laws (add, multiply, divide, invert), generator order
Shamir 3-of-5	Every one of the 10 valid triples reconstructs; no pair ever does; a single share leaks nothing
AES-256-GCM	Round-trip, tamper rejection, AAD binding, key/nonce/tag lengths
Watermark	DCT embed/extract, PSNR, magic detection, Hamming matching
Audit chain	Linkage, verification, tamper detection
Dynamic assembly	Determinism, distinctness across candidates, blueprint counts, no plaintext at rest, release gating, post-seal immutability
Leak-match	Exact-text match, no-match on unrelated text, owner-is-strongest-lead, role enforcement, audit
API flow	Role enforcement, the time gate blocking 3 valid

	shares, legitimate unlock, candidate access, watermark issue & trace
--	--

Run them with: `cd backend && pytest -v`

16. How to Run It Yourself

Backend (API server)

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn trace.api.app:app --reload          # http://127.0.0.1:8000
# interactive API docs at http://127.0.0.1:8000/docs
```

On first start the database is created and demo accounts are seeded (dev-only passwords):
 admin/admin123, investigator/invest123, cust1..cust5 / custodian1..5, cand001/candidate1.

Frontend (dashboards)

```
cd frontend
npm install
npm run dev          # http://localhost:5173 (proxies /api -> backend:8000)
```

Command-line demos (no browser needed)

```
cd backend
python -m cli.crypto_demo      # encrypt -> split -> 3-of-5 reconstruct, 2
                                blocked
python -m cli.api_demo         # time gate holds 3 shares; release; audit
                                tamper caught
python -m cli.watermark_demo   # watermark, JPEG-leak, trace back to the
                                candidate
```

The demo loop in the UI: as Admin seal an exam set to "+30 sec"; as each Custodian submit a share; when 3 are in and the timer hits zero the vault opens; as the Candidate view and download your watermarked paper; as the Investigator upload that file to trace it, then Verify Integrity on the ledger.

17. Build Status (Milestones)

Milestone	Scope	Status
M1	Repo scaffold + crypto core (AES-256-GCM, Shamir 3-of-5, tests, CLI)	Done
M2	Release-time gate, hash-chained audit log, JWT roles, FastAPI endpoints	Done
M3	DCT watermark embed/extract/trace + JPEG-	Done

	robustness test	
M3.5	Watermark API wiring (image + trace + registry + custodian share)	Done
M4	Frontend scaffold + 4 role dashboards wired to backend	Done
Dynamic assembly	Encrypted question bank + per-candidate paper assembly (leak-resistance layer)	Done
Leak-match	Text-leak detector: match to bank + narrow the source	Done
M5	Hero visuals: forensic leak-trace reveal + time-lock vault	In progress
M6	Seed data, one-command demo script, README, architecture diagram	In progress

18. Project Layout

```

trace/
|- backend/
|  |- trace/
|  |  |- crypto/      gf256 . shamir . aes_gcm      <- crypto core
|  |  |- security/    passwords . tokens(JWT) . deps . keywrap
|  |  |- audit/       chain.py  (SHA-256 hash chain)
|  |  |- watermark/   core(DCT) . render . trace    <- watermarking
|  |  |- services/    exams.py . assembly.py (dynamic bank) . leakmatch.py .
watermarking.py
|  |  |- api/         app.py + routers/ (auth . exams . questions .
investigator . audit)
|  |  |- models.py . schemas.py . db.py . config.py . bootstrap.py
|  |- tests/          gf256 . shamir . aes_gcm . watermark . audit_chain .
api . assembly . leakmatch
|  |- cli/            crypto_demo.py . api_demo.py . watermark_demo.py
|- frontend/          React app (Vite + Tailwind + React Query)
|  |- src/
|  |  |- components/  design system + LogView, CountdownTimer, ShareSlots,
VaultState
|  |  |- dashboards/  Admin . Custodian . Candidate . Investigator
|  |  |- api/         client.js . hooks.js (React Query, real endpoints)
|  |  |- auth/        AuthContext (auto-login + role switching)
|- sample_data/       sample_paper.txt
|- scripts/           run_demo.sh, build_docs.py

```

19. Glossary

Term	Plain-language meaning
AES-256-GCM	A strong, authenticated encryption algorithm. "Authenticated" means any tampering with the encrypted data is detected on decryption.

Shamir's Secret Sharing	A way to split a secret into n pieces so that any k of them rebuild it, but fewer than k reveal absolutely nothing.
GF(256)	A finite field of 256 elements — the mathematical playground where the secret-sharing arithmetic happens (the same one AES uses).
Lagrange interpolation	The math used to rebuild the secret from k shares by fitting a polynomial through them.
DCT (Discrete Cosine Transform)	A frequency transform (the heart of JPEG) used here to hide a fingerprint where it survives recompression.
Watermark fingerprint	An invisible code identifying which candidate a paper copy belongs to.
PSNR	A measure of image quality; ~36 dB here means the watermark is invisible to the eye.
Hash chain	A sequence of records each sealed with a SHA-256 hash of the previous one, so any change anywhere is detectable.
Custodian	A trusted official who holds one share of the decryption key.
Time gate	The server rule that refuses decryption until the official release time, no matter what.
JWT	A signed token that proves who a logged-in user is and what role they hold.
Dynamic assembly	Building each candidate's paper on demand from a bank of encrypted questions, so no full paper exists before release.
Blueprint	The recipe for a dynamic exam: which sections, topics, difficulties, and how many questions each.
Containment match	A similarity measure: what fraction of a question's words appear in a leaked text — used to detect text leaks.